

The Great Divide: Binary Search

Grade 6 Computer Science • Lesson Guide & Activity Worksheet

1. Linear Search vs. Binary Search

How do computers search through millions of items in a fraction of a second? Depending on how data is organized, algorithms choose between two main search strategies:

Linear Search (One-by-One)

- **How it works:** Check every single item starting from the beginning until you find the target.
- **Requirement:** Works on *any* list (sorted or unsorted).
- **Analogy:** Searching for a spice jar in a messy market stall or flipping page-by-page from page 1.
- **Speed:** Slow for large datasets (up to N tries for N items).

Binary Search (Divide & Conquer)

- **How it works:** Jump to the middle, check if target is higher/lower, and eliminate half the options!
- **Requirement:** **MUST** be on a **SORTED** list (e.g., numerical or alphabetical).
- **Analogy:** Opening a dictionary right to the middle to locate a word.
- **Speed:** Extremely fast (takes at most ~7 tries for 100 items!).

2. Why Must the List Be Sorted?

If a list is unsorted (e.g., a random list of student names), looking at the middle element tells you nothing about where the other names are located. Binary Search fails completely!

Key Principle

When roll numbers (1–60) or names (A–Z) are sorted, knowing that your target is greater than the middle item guarantees that you can safely discard the entire lower half. **"Sorting is the secret key to Binary Search's speed!"**

3. The 4-Step Binary Search Algorithm & Code

The core algorithm operates in 4 repeated steps:

1

Find the Middle: Locate the middle item of your current sorted list range.

2

Compare with Target: Is the middle item your target? If yes, you found it! If not, check if target is higher or lower.

3

Discard Half: Eliminate/cross out the entire half where your target cannot possibly exist.

4

Repeat: Repeat Steps 1–3 on the remaining half until the target item is found or range is empty.

Python Code Implementation

Here is how programmers write Binary Search in Python using iterative loops:

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2 # Find the middle index

        if arr[mid] == target:
            return mid # Target found! Return index
        elif arr[mid] < target:
            low = mid + 1 # Target is in right half -> discard left half
        else:
            high = mid - 1 # Target is in left half -> discard right half

    return -1 # Target not found in list

# Example Usage:
numbers = [4, 12, 19, 25, 33, 47, 58, 62, 81]
result = binary_search(numbers, 58)
print(f"Target found at index: {result}") # Output: Target found at index: 6
```

4. Real-World Applications

Algorithms like Binary Search power software and services we interact with daily:

- **IRCTC & Railway Reservation Systems:** Searching millions of PNR status records instantly using sorted database indexes.
- **Search Engines (Google):** Rapidly retrieving web pages matching search queries out of billions of indexed documents.
- **Online Dictionaries & Phonebooks:** Finding contact details or word definitions without scanning every entry.

Student Worksheet & Activity

Date: _____

Student Name: _____

Activity 1: Guided Walkthrough — The Library Challenge

Scenario: Our school library has 100 books numbered consecutively from 1 to 100, placed in order on a long shelf. We need to find **Book #72** using Binary Search.

Step #	Current Range	Middle Book Checked	Comparison (Higher/Lower?)	Action Taken
Guess 1	1 to 100	Book #50	Target (#72) is Higher	Discard books 1–50
Guess 2	51 to 100	Book #75	Target (#72) is Lower	Discard books 76–100
Guess 3	51 to 74	Book #62	Target (#72) is Higher	Discard books 51–62
Guess 4	63 to 74	Book #68	Target (#72) is Higher	Discard books 63–68
Guess 5	69 to 74	Book #71	Target (#72) is Higher	Discard books 69–71
Guess 6	72 to 74	Book #73	Target (#72) is Lower	Discard books 73–74
Guess 7	72 to 72	Book #72	Match Found!	Search Complete

Notice: Even in the worst-case scenario for 100 items, Binary Search took only 7 steps, whereas Linear Search could have taken up to 72 steps!

Activity 2: Concept Reinforcement Exercises

Question 1: Unsorted Data Challenge

Suppose you have an unsorted list of 8 animal names: [Zebra, Monkey, Elephant, Cat, Dog, Bear, Lion, Fox].
Can you perform a Binary Search directly to find "Dog"? Explain why or why not, and describe what step must be taken first.

Question 2: Efficiency Comparison Table

Fill in the missing values for maximum number of guesses required in the worst-case scenario:

Total Items in List (N)	Linear Search (Max Guesses)	Binary Search (Max Guesses)
10 items	10 guesses	4 guesses
100 items	100 guesses	7 guesses
1,000 items	1,000 guesses	[____] guesses

Total Items in List (N)	Linear Search (Max Guesses)	Binary Search (Max Guesses)
1,000,000 items	1,000,000 guesses	20 guesses

Question 3: Real-World Scenario Identification

State whether Linear Search or Binary Search is appropriate and why:

a) Finding a contact name in an alphabetically ordered contact list on a smartphone.

b) Finding a lost blue sock in an unsorted pile of laundry.

Practice Questions & Coding Exercises

Grade 6 Assessment & Problem Solving

1. Multiple Choice Questions

a) What is the primary prerequisite requirement before you can run a Binary Search algorithm on a list?

- A) The list must contain only even numbers.
- B) The list must be sorted in order.
- C) The list must have fewer than 50 items.
- D) The target item must be right in the middle.

b) If you double the size of a sorted list from 100 elements to 200 elements, how many additional steps does Binary Search take in the worst case?

- A) 100 extra steps
- B) 50 extra steps
- C) 1 extra step
- D) It doubles the number of steps

2. Binary Search Algorithm Trace

Consider the following sorted array of 9 numbers:

[4, 12, 19, 25, 33, 47, 58, 62, 81]

You are searching for target value **58** using Binary Search.

Task: Fill in the trace table below for each step of the search process:

Step	Search Range (Sub-array)	Middle Value	Higher / Lower?	Eliminated Half
Step 1				
Step 2				
Step 3				

3. Python Code Analysis Task Coding Challenge

Look at the snippet of Python code below and answer the questions:

```
def find_number(arr, target):  
    low = 0  
    high = len(arr) - 1  
    while low <= high:  
        mid = (low + high) // 2  
        if arr[mid] == target:  
            return mid  
        elif arr[mid] < target:  
            low = mid + 1  
        else:  
            high = mid - 1  
    return -1
```

a) What line of code determines the middle element index of the array?

b) What value is returned if the target is *not* present in the array?

4. Real-World Application Challenge

An e-commerce website has 10,000 products stored in its database. When a customer searches for Product ID **#4,521**, explain why the server uses Binary Search instead of Linear Search, and calculate the maximum number of checks the server will perform before finding the product.